push-topology-audit

Push-topology audit — every repository in this tree

Measured 2026-09-05/06 from this repository's root on the development host. Read-only throughout: `git remote -v`, `git remote get-url --all --push`, `git ls-remote`, `gh api`, `glab api`, anonymous `curl`. Nothing was pushed, fetched-and-merged, added, removed or enabled. No remote configuration was changed.

Every figure below is a dated observation, not a standing fact. Repository visibility is a provider setting that changes outside this tree, and three submodule HEADs moved *during* this audit (see The tree was moving). Re-run the commands; do not quote the numbers.

---

0 — Method, and the flag that inverts the answer

`git remote get-url --push <remote>` prints only the **first** push URL. On a remote with four push URLs it returns one, with exit code 0 and no warning. An audit built on it reports the opposite of the truth.

Measured on `monetization`'s `origin`, both forms side by side:

```
git remote get-url --push       origin   ->
git@gitflic.ru:milosvasic/monetization.git          (1 URL)
git remote get-url --all --push origin   ->
git@gitflic.ru:milosvasic/monetization.git

git@github.com:milos85vasic/monetization.git

git@gitlab.com:milos85vasic/monetization.git
```

ssh://
git@gitverse.ru:2222/milosvasic/monetization.git   (4 URLs)

Every push count in this document comes from `--all --push`, cross-checked against the `(push)` lines of `git remote -v` and against the `pushurl` line count in each repository's own `.git/config`. The three agree everywhere.

---

1 — Push topology, all 14 repositories

13 gitlinks are declared in `.gitmodules` (`git config -f .gitmodules --get-regexp 'submodule\..*\.path'`, rc=0), plus the umbrella itself. All 14 are initialised and were probed.

1a — Fan-out remotes (more than one push URL)

Eight of the fourteen repositories carry a remote that publishes to multiple destinations from a single `git push`. In every case the fan-out remote is `origin`.

Repository	Remote	Fetch URL	Push URLs	Fetch host ≠ first push host?
`submodules/constitution`	`origin`	`github.com:HelixDevelopment/HelixConstitution`	**6**	**YES** — fetches github, pushes gitflic first
`monetization`	`origin`	`github.com:milos85vasic/monetization`	**4**	**YES** — fetches github, pushes gitflic first
`submodules/containers`	`origin`	`github.com:vasic-digital/containers`	**3**	no (github first)
`milosvasic.ru`	`origin`		**2**	**YES** — fetches

Repository	Remote	Fetch URL	Push URLs	Fetch host ≠ first push host?
		`github.com:milos85vasic/milosvasic.ru`		github, pushes gitflic first
`submodules/LLMProvider`	`origin`	`github.com:vasic-digital/LLMProvider`	2	no
`submodules/RAG`	`origin`	`github.com:vasic-digital/RAG`	2	no
`submodules/verdict`	`origin`	`github.com:vasic-digital/verdict`	2	no
`submodules/passage`	`origin`	`github.com:vasic-digital/passage`	2	no

That is **eight** fan-out remotes across **eight** repositories — the table above lists every one. No repository has more than one fan-out remote.

Full push-URL sets for the fan-out remotes:

```
submodules/constitution  origin ->  gitflic.ru:helixdevelopment/
helixconstitution
                                    github.com:HelixDevelopment/
HelixConstitution
                                    gitlab.com:helixdevelopment1/
helixconstitution
                                    gitverse.ru:helixdevelopment/
HelixConstitution
                                    github.com:vasic-digital/
HelixConstitution
                                    gitlab.com:vasic-digital/
HelixConstitution

monetization             origin ->  gitflic.ru:milosvasic/
monetization
                                    github.com:milos85vasic/
monetization
                                    gitlab.com:milos85vasic/
monetization
                                    gitverse.ru:2222/milosvasic/
monetization      (ssh://, non-standard port)

submodules/containers    origin ->  github.com:vasic-digital/
Containers
```

```
                                    gitlab.com:vasic-digital/
containers
                                    gitlab.com:vasic-digital/
Containers           <- same GitLab project as the line above

milosvasic.ru            origin ->  gitflic.ru:milosvasic/
milosvasic-net-v-2
                                    github.com:milos85vasic/
milosvasic.net.v2     <- rename alias, see §1c

submodules/LLMProvider   origin ->  github.com:vasic-digital/
LLMProvider + gitlab.com:vasic-digital/LLMProvider
submodules/RAG           origin ->  github.com:vasic-digital/
RAG        + gitlab.com:vasic-digital/RAG
submodules/verdict       origin ->  github.com:vasic-digital/
verdict    + gitlab.com:vasic-digital/verdict
submodules/passage       origin ->  github.com:vasic-digital/
passage    + gitlab.com:vasic-digital/passage
```

1b — Single-destination repositories

Repository	Remotes	Push destinations
. (umbrella)	github, origin, upstream	all three → github.com:milos85vasic/vasic (1 URL each)
vasic.digital	github, origin, upstream	all three → github.com:vasic-digital/vasic-digital.github.io
design-toolkit	github, origin, upstream	all three → github.com:vasic-digital/design-toolkit
ai_interviewing	github, origin, upstream	all three → github.com:milos85vasic/ai_interviewing
workshop	github, origin, upstream	all three → github.com:milos85vasic/workshop_curriculum
submodules/superspec	origin	github.com:WangX0111/superspec (third-party upstream)

`design-toolkit` declares NO `gitlab` remote in this checkout — `git remote -v` returns `github`, `origin`, `upstream`, all GitHub. `CLAUDE.md` records that a `gitlab` remote *is* declared there; that is **no longer true of this checkout** (the gitlink has since moved to `a135aa8eebb1`). The mirror is still measurable by probing the URL directly — see §6.

1c — Three push URLs are rename aliases, not distinct repositories

Confirmed by `gh api repos/<x> --jq .full_name` (rc=0) and by `ls-remote` returning the same SHA through both names:

Push URL as configured	Resolves to	Note
`github.com:milos85vasic/milosvasic.net.v2`	`milos85vasic/milosvasic.ru`	**the live production site repo**
`github.com:HelixDevelopment/HelixConstitution`	`HelixDevelopment/constitution`	governance source
`github.com:vasic-digital/Containers`	`vasic-digital/containers`	case-only difference

This matters for `milosvasic.ru`: `origin`'s push set does **not** literally contain `milos85vasic/milosvasic.ru`, so a naive reading says `git push origin` never reaches production. It does — through GitHub's rename redirect. **That redirect is a provider convenience, not a guarantee: GitHub retires it if the old name is ever reclaimed.** The publish path of a live production site currently depends on it.

Likewise `submodules/containers` `origin` pushes to the **same GitLab project twice** (`containers` and `Containers` both resolve to `vasic-digital/containers`, `glab api` rc=0), so one of its three push URLs is redundant.

---

2 — Visibility, by measurement

GitHub — `gh api repos/<r> --jq '[.full_name,.visibility,(.private|tostring),.default_branch,.pushed_at]'`, all rc=0

Repository	Visibility	Default branch
`milos85vasic/vasic` (umbrella)	**public**	main
`HelixDevelopment/constitution`	**public**	main
`vasic-digital/HelixConstitution`	**PRIVATE**	main
`milos85vasic/milosvasic.ru`	**public**	main
`vasic-digital/vasic-digital.github.io`	**public**	main
`vasic-digital/design-toolkit`	**public**	main
`milos85vasic/ai_interviewing`	**PRIVATE**	main
`milos85vasic/monetization`	**PRIVATE**	main
`milos85vasic/workshop_curriculum`	**PRIVATE**	main
`WangX0111/superspec`	**public**	main
`vasic-digital/containers`	**public**	main
`vasic-digital/LLMProvider`	**public**	master
`vasic-digital/RAG`	**public**	main
`vasic-digital/verdict`	**public**	main
`vasic-digital/passage`	**public**	main

The three private repositories named in the audit brief are **confirmed private**: `workshop`, `ai_interviewing`, `monetization`. A **fourth** private repository was found that the brief did not name: **`vasic-digital/HelixConstitution` on GitHub**, which is a push target of `submodules/constitution`.

GitLab — `glab api projects/<enc>`, all rc=0

Project	Visibility
`helixdevelopment1/helixconstitution`	public
`vasic-digital/HelixConstitution`	**public**
`milos85vasic/monetization`	**PRIVATE**

Project	Visibility
`vasic-digital/containers`	public
`vasic-digital/LLMProvider`	public
`vasic-digital/RAG`	public
`vasic-digital/verdict`	public
`vasic-digital/passage`	public
`vasic-digital/design-toolkit`	**PRIVATE**

Inconsistency worth naming: the mirror `vasic-digital/HelixConstitution` is **private on GitHub and public on GitLab**. The same content, the same name, two opposite settings. Its source (`HelixDevelopment/constitution`) is public, so this is not a disclosure — but one of the two settings does not express whatever the intent was.

The two hosts with no API adapter

```
command -v gitflic   ->  (absent)
command -v gitverse  ->  (absent)
```

No read-only API adapter is registered for `gitflic.ru` or `gitverse.ru`, which is the same class the umbrella's own `scripts/verify-provider-ci.sh` reports as UNVERIFIED (6 rows). **This is a missing-adapter problem, not a network problem, and the distinction is measured:** `git ls-remote` over SSH succeeded against both hosts (rc=0, real SHAs returned — see §6), and anonymous HTTPS against both hosts returned HTTP 200 for a public project. Both hosts are reachable from here. What is missing is a way to *ask them a visibility question*.

---

3 — The matrix that matters: private source × push target

For every repository whose own GitHub origin is **private**, every destination a `git push origin` would reach:

Private source	Push target	Host	Adapter?	Visibility of target
`monetization`	`gitflic.ru:milosvasic/monetization`	gitflic.ru	**NO**	**not anonymously readable** (§3a)

Private source	Push target	Host	Adapter?	Visibility of target
				— setting UNDETERMINE
	`github.com:milos85vasic/monetization`	github.com	yes (`gh`)	**private ✔**
	`gitlab.com:milos85vasic/monetization`	gitlab.com	yes (`glab`)	**private ✔**
	`gitverse.ru:2222/milosvasic/monetization`	gitverse.ru	**NO**	**not anonymously readable** (§3a) — setting UNDETERMINE
`ai_interviewing`	`github.com:milos85vasic/ai_interviewing`	github.com	yes	**private ✔**
`workshop`	`github.com:milos85vasic/workshop_curriculum`	github.com	yes	**private ✔**

`ai_interviewing` and `workshop` publish to exactly one destination each, and that destination is confirmed private. **`monetization` is the only private repository in this tree that fans out, and it is the only one with targets this checkout cannot fully classify.**

3a — What the two unadaptered targets *were* measured to be

Absent an API, an anonymous read is still a real measurement, and it was run with **positive controls on both hosts** so that a failure cannot be confused with a broken method. `GIT_TERMINAL_PROMPT=0`, empty credential helper, no credentials offered:

Probe	**`git ls-remote` (anon HTTPS)**	**Web page (anon `curl -L`)**
control `github.com/vasic-digital/design-toolkit` (known public)	rc=0, SHA returned	—
control `github.com/milos85vasic/monetization` (known private)	rc=128, auth challenge	—

Probe	`git ls-remote` (anon HTTPS)	Web page (anon `curl -L`)
control `gitflic.ru/project/helixdevelopment/helixconstitution` (public source)	**rc=0, SHA returned**	**HTTP 200**
control `gitverse.ru/helixdevelopment/HelixConstitution` (public source)	**rc=0, SHA returned**	**HTTP 200**
`gitflic.ru/project/milosvasic/monetization`	rc=128, auth challenge	HTTP 404
`gitverse.ru/milosvasic/monetization`	rc=128, auth challenge	HTTP 404
`gitflic.ru/project/milosvasic/milosvasic-net-v-2`	rc=128, auth challenge	HTTP 404
`gitflic.ru/project/<nonexistent>` (negative control)	—	HTTP 404
`gitverse.ru/<nonexistent>` (negative control)	—	HTTP 404

What this establishes: the probe method works on both hosts (a public project there is anonymously clonable and returns 200), and **neither `monetization` copy is readable by an anonymous stranger.** That is the reassuring half, and it is a genuine measurement rather than an assumption.

What this does NOT establish, and must not be reported as if it did:

1. A 404 is what these hosts return for *both* "private" and "does not exist" — the negative control proves the two are indistinguishable from outside. The repositories **do** exist (SSH `ls-remote` reaches them, §6), so "not anonymously readable" is the finding; "private" is an inference.
2. **Anonymous-unreadable is not the same as private.** Neither host was asked for its visibility *setting*. A project set to an "internal"/"logged-in users only" tier — if these hosts offer one —

would present exactly this way while being readable by every registered user of the platform. **Nothing in this checkout can tell those two states apart.** That is the residual risk, and it is unclosed.
3. Nothing was established about **forks, mirrors or organisation-level sharing** on either host.

Verdict for §3: UNDETERMINED, narrowed but not closed. Two push targets of a private repository sit on hosts whose visibility model this checkout cannot query. Anonymous access is measurably denied today; the setting behind that denial has never been read by anyone here.

---

4 — The reverse check: any private source pushing to a confirmed-public target?

This was checked exhaustively — every push URL of every repository, against the measured visibility table in §2.

Result: NONE. No confirmed private→public flow exists in this tree today.

The four cases that had to be ruled out individually, because each *looks* like one:

Case	Source	Target	Verdict
`monetization` → GitLab	private	`gitlab.com:milos85vasic/monetization` **private**	not a leak
`constitution` → `vasic-digital/HelixConstitution` on GitLab	**public** source (`HelixDevelopment/constitution`)	public	not a leak — public content to a public mirror
`constitution` → `vasic-digital/HelixConstitution` on GitHub	public source	**private** target	public→private, harmless
`milosvasic.ru` → gitflic	public source	not anonymously readable	public→non-public, harmless

Note the direction of the last two: this tree's cross-visibility flows all run **public into private**, which loses nothing. The dangerous direction is empty.

Honest boundary: this is a statement about **configured push destinations**, not about repository *contents*. Whether private material has previously been committed into a public repository is a different question with its own open incident — `docs/content-boundary-incident-2026-09-01.md` and `scripts/verify-content-boundary.sh`, which exits 1 by design. **A clean §4 here does not clear that.**

---

5 — Secret hygiene in provider API responses

The GitLab project API returns a live **runner registration token** in the project payload for callers with sufficient role. Anyone who logs, pipes, pastes or screenshots raw `glab api projects/...` output captures it.

All nine GitLab-hosted projects in this tree were checked. **Values were never read, recorded, or written anywhere — only presence, type and length.**

GitLab project	`runners_token`	Source repo visibility
`milos85vasic/monetization`	**PRESENT** — type `str`, length **29**	private
`helixdevelopment1/helixconstitution`	absent	public
`vasic-digital/HelixConstitution`	absent	public (GitLab)
`vasic-digital/containers`	absent	public
`vasic-digital/LLMProvider`	absent	public
`vasic-digital/RAG`	absent	public
`vasic-digital/verdict`	absent	public
`vasic-digital/passage`	absent	public
	absent	private (GitLab)

GitLab project	`runners_token`	Source repo visibility
`vasic-digital/design-toolkit`		

Exactly one project of nine exposes it, and it is the one private repository with the widest push fan-out. The eight that do not expose it are all in the `vasic-digital` and `helixdevelopment1` group namespaces; the one that does is in the user's **own personal namespace**, where the account holds Owner rather than a group-delegated role. The exposure tracks the caller's role on the project, not any per-project setting made deliberately.

Does it need to be there? Measured alongside it: `shared_runners_enabled=true`, `jobs_enabled=true`, `runner_token_expiration_interval=None` (**no expiry**), and `ci_job_token_scope_enabled=false` on **every** project including this one. No project-level runner is evidenced by these fields. **A never-expiring registration token is being served in an API response for a private repository whose CI posture shows no sign of needing a project runner.** Whether a runner is genuinely registered was not determined — that needs a `runners` listing, which was not run.

No other credential-shaped value was found in any payload. The other keys matching token/secret/key are configuration booleans and claim lists (`ci_job_token_scope_enabled`, `ci_id_token_sub_claim_components`, `runner_token_expiration_interval`, `ci_push_repository_for_job_token_allowed`), not secrets. A separate scan for credentials embedded in remote URLs (`https://user:pass@host` form) across all 14 repositories returned **zero** matches.

---

6 — Mirror sync, every reachable push target

`git ls-remote <push-url> refs/heads/<local-branch>` against the local HEAD, for all 29 distinct push URLs across the 14 repositories. **Every probe returned rc=0 — including both `gitflic.ru` and `gitverse.ru`, over SSH.** No target was unreachable; no auth was refused; nothing timed out.

Result: 29 of 29 configured push targets IN-SYNC with their local HEAD.

That includes every one of the six constitution mirrors (gitflic, two GitHub orgs, two GitLab namespaces, gitverse — all at `cbcf4eebdf0a`), all four `monetization` mirrors (all at `54ed7b0f5add`), and both `milosvasic.ru` targets.

The one mirror that is behind is the one no longer wired up

`design-toolkit`'s GitLab mirror is **not** a configured push target in this checkout (§1b), so it does not appear in the 29. Probing the URL directly — read-only, no remote added:

```
local HEAD
a135aa8eebb17917d699ab3ca3f3c48e99e13d89
gitlab.com:vasic-digital/design-toolkit main
520c436c2c2a33ed3976856463347519b3a710d9   rc=0
merge-base --is-ancestor 520c436c a135aa8e    rc=0     -> pure
lag, no divergence
rev-list --left-right --count 520c436c...a135aa8e  ->  0   7
```

Confirmed: 0 divergent, 7 behind. The brief's figure of 7 is correct and **supersedes the 6 recorded in `CLAUDE.md`** — the GitHub side has advanced one further commit since that reading; the GitLab side has not moved at all (`520c436c` is the same commit both readings name).

The gap is structurally frozen and cannot close by itself: the submodule's push recipe is `upstreams/gitlab.sh.disabled`, and this checkout declares no `gitlab` remote for it. **The mirror is behind a repository that is public on GitHub, so every commit it lacks is already published; the 7-commit gap carries no content-boundary risk.**

No other repository was found lagging any of its mirrors.

The tree was moving

Three submodule HEADs changed between the first and second measurement pass of this audit, consistent with other agents working in the same checkout:

```
submodules/containers    6d13ad03528c  ->  7f5922563d8b
submodules/LLMProvider   e2c6b7bf039c  ->  4c73c8b0dfc6
submodules/RAG           13f4bacd6051  ->  2dffc735f3d8
```

All three still measured IN-SYNC against every mirror at the later reading. **Treat the per-SHA rows as a snapshot, not a baseline** — the sync verdicts are what survive.

---

7 — The blind spot: no gate in this tree reads a push URL

`scripts/verify-submodule-remote-sync.sh` — the only instrument here that looks at a remote at all — reads the URL declared in **`.gitmodules`** (line 309, `while IFS=$'\t' read -r path url`), which is a single *fetch* URL per submodule. It never enumerates configured remotes and never asks for push URLs.

```
git grep -n -- '--all --push'     ->  no matches anywhere in the umbrella
```

Consequence, stated plainly: a submodule could acquire a fifth push URL pointing at any host on earth, and `verify-submodule-remote-sync.sh`, `verify-manifest-pins.sh`, `verify-governance-cascade.sh` and `verify-content-boundary.sh` would all stay exactly as green as they are now. The entire fan-out described in §1 is invisible to every gate in this repository. It was found by hand, and only because someone went looking.

The fan-out mechanism is also visible in the tracked `upstreams/` recipe sets, which agree with the configured push URLs repository by repository:

Repository	`upstreams/` recipes	push URLs
`submodules/constitution`	6	6
`monetization`	4 (`GitFlic`, `GitHub`, `GitLab`, `GitVerse`)	4
`submodules/containers`	4	3
`submodules/LLMProvider`	4	2
`submodules/RAG`	2	2
`submodules/verdict`, `submodules/passage`	2 each	2 each
`design-toolkit`	2, one `.disabled`	1
`.` (umbrella), `ai_interviewing`, `workshop`	1 each	1 each

Repository	`upstreams/` recipes	push URLs
`milosvasic.ru`, `vasic.digital`	**none**	2, 1

`milosvasic.ru` has **no `upstreams/` directory at all** yet carries two push URLs — its fan-out lives only in `.git/config`, which is untracked. A fresh clone reproduces neither.

---

8 — Everything UNDETERMINED

Recorded as 2, never as a pass.

1. **Visibility *setting* of `gitflic.ru:milosvasic/monetization`.** No API adapter. Anonymous read denied and web page 404 (with working positive controls on that host), so it is not open to strangers — but "not anonymously readable" is not "private", and an internal/logged-in-users tier would be indistinguishable from here.
2. **Visibility *setting* of `gitverse.ru:2222/milosvasic/monetization`.** Identical situation, identical evidence, identical residual gap.
3. **Visibility of `gitflic.ru:milosvasic/milosvasic-net-v-2`.** Same class. Source repo is public, so the stakes are lower, but the answer is equally unknown.
4. **Whether a GitLab runner is actually registered against `milos85vasic/monetization`** — i.e. whether the never-expiring `runners_token` serves any purpose. Not probed; needs a `runners` listing.
5. **Whether the `milosvasic.net.v2` rename redirect will persist.** Provider behaviour, unreadable from here. Today it resolves; a production publish path depends on it.
6. **Whether the two unadaptered hosts expose these repositories through forks, mirrors, or org-level sharing.** Not probed by any method available here.
7. **Whether `.git/config` in a fresh clone would reproduce any of this topology.** The push fan-out lives in untracked local config; only the `upstreams/` recipes are tracked, and they do not match the push URLs everywhere (§7).

---

9 — What an operator must decide

Nothing below was acted on. Each item names its cost.

1. **Get a read-only visibility answer for `gitflic.ru` and `gitverse.ru`, or accept permanent ignorance about two destinations of a private repository.** Both hosts are reachable and both serve public projects anonymously, so the gap is genuinely *adapter-shaped* and closable — a small read-only API client for each, registered the way `gh`/`glab` are, would move items 1–3 above out of UNDETERMINED for good. Cost: real work against two APIs with no existing tooling. Doing nothing costs nothing today and leaves a private repository publishing to two hosts nobody has ever asked a question of.

2. **Decide whether `monetization` should fan out to four hosts at all.** It is the only private repository in the tree that does, and its two unverifiable destinations are the *entire* content of §3. Reducing `origin`'s push set to the two hosts that can be audited would close the finding outright. Cost: the Russian-host mirrors stop receiving commits, which may be exactly why they exist. **This is a hosting-policy decision, not a security defect** — the audit found no evidence of exposure, only of unverifiability.

3. **Rotate the GitLab runner registration token on `milos85vasic/monetization`, or establish that it is needed.** It has no expiry, it is served in an API response for a private repository, and no project runner is evidenced. Cost of rotating: any genuinely registered runner must be re-registered. Cost of leaving it: a permanent credential in every raw `glab api` transcript for that project. **Determine first (item 4 in §8), then rotate — do not rotate blind.**

4. **Reconcile `vasic-digital/HelixConstitution`: private on GitHub, public on GitLab.** Two opposite settings on the same mirrored content. Harmless today because the source is public; it is a signal that one of the two was set without the other in view.

5. **Decide whether `milosvasic.ru`'s production publish path should keep depending on a GitHub rename redirect.** `origin` pushes to `milosvasic.net.v2`, which resolves to `milosvasic.ru` only because GitHub still honours the old name. Pointing the push URL at the current name is a one-line config change with no downside. Cost of leaving it: a live site whose deploy path breaks silently if the old name is ever reclaimed.

6. **Teach a gate to read push URLs.** Every finding in this document was invisible to every instrument in this repository (§7). A check that enumerates `git remote get-url --all --push` per repository, resolves each host, and fails when a private repository gains a destination that cannot be classified would make this audit repeatable instead of one-off. Cost: another gate to maintain — and it should be paired-mutation proved like the rest, or it is worth nothing.

7. **Drop the redundant duplicate push URL on `submodules/containers`** (`Containers` and `containers` are the same GitLab project). Cosmetic; costs nothing either way.

Recommended ordering: 5 (free, protects production), then 3 (after determining 4 in §8), then 1, then 2, then 6. Items 4 and 7 are tidy-ups.

---

10 — Re-derive; do not quote this file

```
# topology — the ONLY correct flag
for d in . $(git config -f .gitmodules --get-regexp
        'submodule\..*\.path' | awk '{print $2}'); do
  echo "## $d"
  for r in $(git -C "$d" remote); do
    echo "  $r fetch: $(git -C "$d" remote get-url --all
        "$r" | tr '\n' ' ')"
    echo "  $r push : $(git -C "$d" remote get-url --all --push
        "$r" | tr '\n' ' ')"
  done
done

# visibility
gh   api repos/<owner>/<name>      --jq '[.full_name,.visibility]|
        join(" ")'
glab api projects/<owner>%2F<name> | python3 -c 'import
        sys,json;d=json.load(sys.stdin);print(d["path_with_namespace"],d["visibil

# anonymous readability on a host with no adapter (positive
        control first!)
GIT_TERMINAL_PROMPT=0 git -c credential.helper= ls-remote https://
        <host>/<path>.git HEAD

# mirror sync
git -C <repo> ls-remote <push-url> refs/heads/<branch>
```